

Laboratorio de Java / Practica 3

MPOO

Esquivel Cortes Iker Alonso

Hernandez Crisostomo Emilio

Bloque 1

```
=====
BLOQUE 1 • Con que nace un atributo al que nadie le puso valor
Estos atributos se declararon, pero nunca se les asigno nada.
=====
```

Asi estan declarados, fijate en el camelCase de cada nombre:

```
int    enteroSinValor;
double decimalSinValor;
boolean logicoSinValor;
char   letraSinValor;
String textoSinValor;
```

Que imprime cada uno?

```
enteroSinValor  = 0
decimalSinValor = 0.0
logicoSinValor  = false
(int) letraSinValor = 0
textoSinValor   = null
```

+-----+-----+-----+-----+				
RESULTADO DEL BLOQUE 1				
+-----+-----+-----+-----+				
expresion	predijiste	salio		
+-----+-----+-----+-----+				
enteroSinValor	0	0	ok	
decimalSinValor	0.0	0.0	ok	
logicoSinValor	false	false	ok	
(int) letraSinValor	0	0	ok	
textoSinValor	null	null	ok	
+-----+-----+-----+-----+				

Sin fallos en este bloque.

Bloque 2

=====

BLOQUE 2 · La division que miente

Fijate en el tipo de cada operando y en donde esta puesto el casting.

=====

Que imprime cada expresion?

7 / 2 = 3

7.0 / 2 = 3.5

(double) 7 / 2 = 3.5

(double) (7 / 2) = 3.5

7.0 / 0 = null

0.0 / 0.0 = infinity

+-----+-----+-----+-----+				
RESULTADO DEL BLOQUE 2				
+-----+-----+-----+-----+				
expresion	predijiste	salio		
+-----+-----+-----+-----+				
7 / 2	3	3	ok	
7.0 / 2	3.5	3.5	ok	
(double) 7 / 2	3.5	3.5	ok	
(double) (7 / 2)	3.5	3.0	FALLO	
7.0 / 0	null	Infinity	FALLO	
0.0 / 0.0	infinity	NaN	FALLO	
+-----+-----+-----+-----+				

Fallaste 3. Anotalas: explicarlas es lo que se califica.

//Comentarios de Errores

(double)(7/2):

Porque creímos que funcionaba como si se declaraba un valor flotante y afectaba el uso de los paréntesis con la sintaxis y el operador

7.0/0:

Porque en matemáticas clásicas una división entre cero es una indeterminación, por eso creímos que se imprimía un valor null

0.0/0.0:

No sabíamos que existia el Nan (Not a number), y nos guiamos en la teoría de límites cuando indica que esa división tiende a infinito

Bloque 3

```
=====
BLOQUE 3 · El modulo, y para que sirve de verdad
El % da el residuo: sirve para saber si algo es par y para sacar digitos.
=====
```

Que imprime cada expresion?

```
7 % 2 = 1
10 % 5 = 0
-7 % 2 = 1
7 % 2.5 = 0
12345 % 10 = 5
12345 / 10 = 1234
```

+-----+-----+-----+-----+			
RESULTADO DEL BLOQUE 3			
+-----+-----+-----+-----+			
expresion	predijiste	salio	
+-----+-----+-----+-----+			
7 % 2	1	1	ok
10 % 5	0	0	ok
-7 % 2	1	-1	FALLO
7 % 2.5	0	2.0	FALLO
12345 % 10	5	5	ok
12345 / 10	1234	1234	ok
+-----+-----+-----+-----+			

Fallaste 2. Anotalas: explicarlas es lo que se califica.

//

-7%2:

Porque pensamos que el signo negativo no afectaba al residuo de la división

7%2.5:

Porque hubo una confusión en calcular los decimales para obtener el residuo

Bloque 4

```
=====
BLOQUE 4 · El casting y lo que se pierde
Un casting no redondea: corta. Y un tipo chico no aguanta cualquier numero.
=====
```

Que imprime cada expresion?

```
(int) 3.9 = 3
(int) -3.9 = -3
(char) 65 = A
(int) 'A' = 65
'A' + 1 = 66
(char) ('A' + 1) = B
(byte) 200 = 2
```

RESULTADO DEL BLOQUE 4			
expresion	predijiste	salio	
(int) 3.9	3	3	ok
(int) -3.9	-3	-3	ok
(char) 65	A	A	ok
(int) 'A'	65	65	ok
'A' + 1	66	66	ok
(char) ('A' + 1)	B	B	ok
(byte) 200	2	-56	FALLO

Fallaste 1. Anotalas: explicarlas es lo que se califica.

//

(byte)200:

Porque creíamos que se refiere al valor de los bytes en un decimal menos, pero en realidad era el valor de 200 en bytes, o el valor de un carácter

Bloque 5

=====

BLOQUE 5 · Relacionales y logicos

El && se detiene en cuanto ya sabe la respuesta: eso es el corto circuito.

=====

Aqui vidasRestantes vale 0. Que imprime cada expresion?

5 > 3 = true

5 == 5.0 = true

0.1 + 0.2 == 0.3 = false

true && false = false

true || false = true

vidasRestantes > 0 && 10 / vidasRestantes > 1 = false

+-----+-----+-----+-----+				
RESULTADO DEL BLOQUE 5				
+-----+-----+-----+-----+				
expresion	predijiste	salio		
+-----+-----+-----+-----+				
5 > 3	true	true	ok	
5 == 5.0	true	true	ok	
0.1 + 0.2 == 0.3	false	false	ok	
true && false	false	false	ok	
true false	true	true	ok	
vidasRestantes > 0 && 10 / vi~	false	false	ok	
+-----+-----+-----+-----+				

Sin fallos en este bloque.

//

Bloque 6

BLOQUE 6 · Las sorpresas

Aquí es donde casi todos fallan. Por eso existe este laboratorio.

Que imprime cada expresion?

`0.1 + 0.2 = 0.3`

`Integer.MAX_VALUE + 1 = 1`

`5 + 3 = 8`

`"5" + 3 = 56`

`1 + 2 + "3" = 54`

`"1" + 2 + 3 = 54`

RESULTADO DEL BLOQUE 6			
expresion	predijiste	salio	
<code>0.1 + 0.2</code>	<code>0.3</code>	<code>0.300000~</code>	FALLO
<code>Integer.MAX_VALUE + 1</code>	<code>1</code>	<code>-2147483~</code>	FALLO
<code>5 + 3</code>	<code>8</code>	<code>8</code>	ok
<code>"5" + 3</code>	<code>56</code>	<code>53</code>	FALLO
<code>1 + 2 + "3"</code>	<code>54</code>	<code>33</code>	FALLO
<code>"1" + 2 + 3</code>	<code>54</code>	<code>123</code>	FALLO

Fallaste 5. Anotalas: explicarlas es lo que se califica.

//

0.1+0.2 :

Olvidamos que el código estaba incluido con mas decimales, partiendo a infinito, pero solo incluimos un decimal

Integer.MAX_VALUE+1 :

No entendimos la función de MaxValue, ahora entendemos que dentro del codigo la funcion integer se refiere a ava sirve para envolver un valor numérico entero (int) dentro de un objeto, lo que permite usar métodos útiles y almacenar el valor null

"5"+3 :

Creímos que "5" se refiere al valor en ascii de ese símbolo, pero nos dimos cuenta que no se refería a un char

1+2+"3" :

Lo mismo, creemos que se refería a un char y en valor de ascii pero era solo el valor agregado

“1”+2+3 :

Creíamos que otra vez se refería al valor de ascii de char 1 más los decimales pero no lo era, supimos que se refería a otro tipo de valor

Bloque 7

```
=====
BLOQUE 7 · AHORA TU (aquí escribes código, no predicciones)
=====
7.1 = 2.5
7.2 = false
7.3 = 9
7.4 = true
7.5 = false
(si todavía no escribes nada aquí, no se imprime nada: es normal)
```

TABLA FINAL

#####				
#	TU TABLA FINAL · ESTA ES LA CAPTURA QUE TIENES QUE ENTREGAR	#		
#####				
#	expresion	predijiste	salio	

1	enteroSinValor	0	0	ok
2	decimalSinValor	0.0	0.0	ok
3	logicoSinValor	false	false	ok
4	(int) letraSinValor	0	0	ok
5	textoSinValor	null	null	ok
6	7 / 2	3	3	ok
7	7.0 / 2	3.5	3.5	ok
8	(double) 7 / 2	3.5	3.5	ok
9	(double) (7 / 2)	3.5	3.0	FALLO
10	7.0 / 0	null	Infinity	FALLO
11	0.0 / 0.0	infinity	NaN	FALLO
12	7 % 2	1	1	ok
13	10 % 5	0	0	ok
14	-7 % 2	1	-1	FALLO
15	7 % 2.5	0	2.0	FALLO
16	12345 % 10	5	5	ok
17	12345 / 10	1234	1234	ok
18	(int) 3.9	3	3	ok
19	(int) -3.9	-3	-3	ok
20	(char) 65	A	A	ok
21	(int) 'A'	65	65	ok
22	'A' + 1	66	66	ok
23	(char) ('A' + 1)	B	B	ok
24	(byte) 200	2	-56	FALLO
25	5 > 3	true	true	ok
26	5 == 5.0	true	true	ok
27	0.1 + 0.2 == 0.3	false	false	ok
28	true && false	false	false	ok
29	true false	true	true	ok
30	vidasRestantes > 0 && 10 / vida~	false	false	ok
31	0.1 + 0.2	0.3	0.30000000~	FALLO
32	Integer.MAX_VALUE + 1	1	-2147483648	FALLO
33	5 + 3	8	8	ok
34	"5" + 3	56	53	FALLO
35	1 + 2 + "3"	54	33	FALLO
36	"1" + 2 + 3	54	123	FALLO

ACERTASTE 25 DE 36				